

PLATEFORME DE MATCHING PROFESSIONNEL

Dossier de cadrage conceptuel et technologique

Une mise en relation candidat-recruteur fondée sur un intérêt réciproque, puis prolongée par une messagerie sécurisée.

PÉRIMÈTRE	STACK VALIDÉE	STATUT
MVP web à deux rôles	React + Node + MongoDB	Cadrage consolidé

Document de synthèse destiné à la présentation et à l'alignement de l'équipe.

Version 1.0 - Septembre 2026

0. Synthèse exécutive

Le résultat consolidé des décisions conceptuelles et technologiques prises en équipe.

Le projet vise un MVP simple, démontrable et défendable : un candidat manifeste son intérêt pour une offre ; le recruteur évalue uniquement les candidats intéressés ; un intérêt réciproque crée un match et ouvre une conversation.

Décisions structurantes

Dimension	Décision retenue	Justification
Produit	Plateforme de mise en relation professionnelle par intérêt réciproque	Identité claire et parcours plus fluide qu'une candidature classique.
MVP	Deux rôles, profils, offres, intérêt candidat, sélection recruteur, match et messagerie	Couvre une boucle métier complète sans multiplier les fonctions secondaires.
Frontend	React.js	Interfaces dynamiques et composants réutilisables pour les cartes et tableaux de bord.
Backend	Node.js + Express.js	Architecture lisible, API REST et intégration naturelle avec Socket.IO.
Données	MongoDB + Mongoose	Souplesse des profils et validation explicite des documents.
Temps réel	Socket.IO + MongoDB	Transmission immédiate et persistance durable des messages.
CV	Multer + stockage disque pour le MVP	Réception contrôlée des PDF ; MongoDB conserve uniquement les métadonnées.
IA	Aucune fonction IA dans le MVP	Priorité à la maîtrise du socle ; compatibilité candidat-offre envisageable en V2.

Principe directeur

Le module évalue autant la démarche, la compréhension et la capacité d'explication que le résultat fonctionnel. L'équipe privilégie donc une architecture limitée mais maîtrisée, avec une progression du minimum viable vers les enrichissements.

1. Vision et proposition de valeur

Une expérience de recrutement centrée sur la réciprocité et la qualité de la mise en relation.

Problème adressé

Les plateformes classiques placent souvent le candidat dans une logique de dépôt de CV sans retour immédiat, tandis que le recruteur reçoit un volume difficile à qualifier. Le projet cherche à rendre l'intérêt explicite des deux côtés avant d'ouvrir un échange.

Proposition de valeur

Pour le candidat	Pour le recruteur	Pour les deux
Découvrir rapidement des offres, exprimer un intérêt et suivre ses matchs.	Publier une offre puis examiner uniquement les profils déjà intéressés.	N'ouvrir une conversation qu'après validation réciproque.

Positionnement verbal

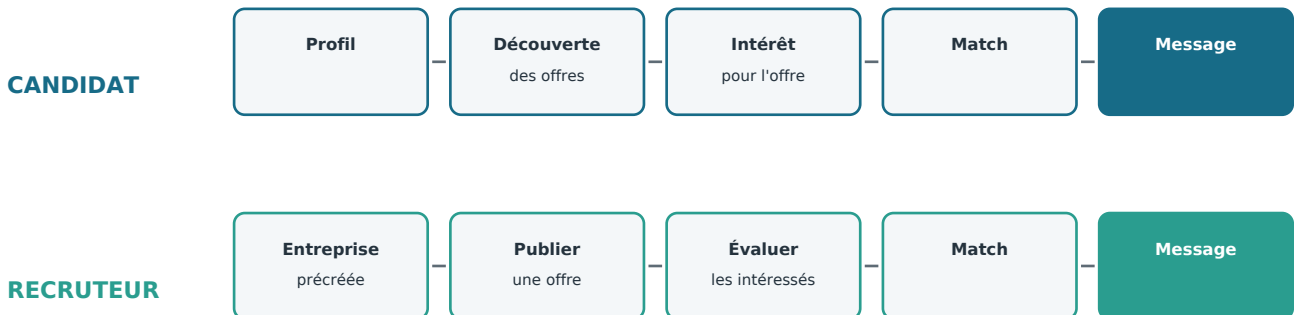
En présentation, le produit sera décrit comme une plateforme de matching professionnel fondée sur l'intérêt réciproque. La comparaison avec une application de rencontre peut servir d'analogie initiale, mais le vocabulaire principal reste professionnel : intérêt, sélection, match, mise en relation et conversation.

Promesse produit

Moins de candidatures envoyées dans le vide ; moins de profils non pertinents à traiter ; une conversation ouverte seulement lorsque les deux parties souhaitent avancer.

2. Acteurs et parcours utilisateur

Deux expériences distinctes convergent vers un seul événement métier : le match.



Acteur 1 - Candidat

- Crée un compte et complète son profil professionnel.
- Ajoute ses compétences, expériences, formations, langues et son CV PDF.
- Découvre les offres publiées sous forme de liste ou de cartes.
- Manifeste un intérêt positif ou passe à l'offre suivante.
- Accède à la conversation uniquement après un match.

Acteur 2 - Recruteur

- Crée un compte rattaché à une entreprise déjà présente en base.
- Publie et ferme ses offres ; aucun brouillon n'est requis dans le MVP.
- Consulte, offre par offre, les candidats ayant déjà manifesté leur intérêt.
- Accepte ou écarte un profil ; l'acceptation réciproque produit le match.
- Échange avec le candidat dans la conversation issue de ce match.

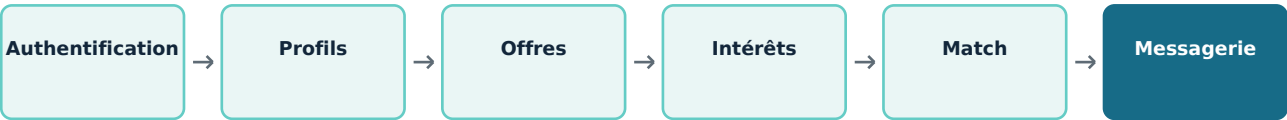
Point UX majeur

Le candidat découvre les offres ; le recruteur découvre les candidats intéressés par une offre précise. Les deux parcours ne sont donc pas symétriques, mais ils sont complémentaires.

3. Périmètre fonctionnel du MVP

Une boucle fonctionnelle complète, avec des exclusions volontaires pour maîtriser le délai.

Chaîne de valeur minimale



Dans le MVP	Reporté en V2	Hors périmètre initial
Inscription, connexion et rôles	Compatibilité candidat-offre	Administration complète
Profil candidat et CV PDF	Notifications avancées	Validation documentaire d'entreprise
Profil recruteur et entreprise précréée	Stockage cloud des CV	Paieement ou abonnement
Publication et fermeture d'offre	Statuts brouillon, archivé, planifié	Visioconférence intégrée
Intérêt candidat et décision recruteur	Recherche et filtres enrichis	Réseau social complet
Match et conversation	Accusés de lecture, saisie en cours	IA générative dans le produit

Critères d'acceptation globaux

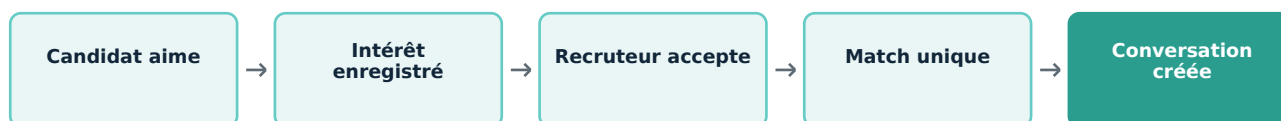
- Un candidat ne peut manifester son intérêt qu'une fois pour une même offre.
- Un recruteur ne voit que les candidatures liées à ses propres offres.
- Un match n'existe que lorsque les deux intérêts sont positifs.
- Une conversation est unique par match et inaccessible aux tiers.
- Une action interdite par le rôle ou la propriété d'une ressource retourne un refus explicite.

4. Règles métier essentielles

Le backend demeure l'autorité : l'interface ne décide jamais seule de l'accès ou de la création d'un match.

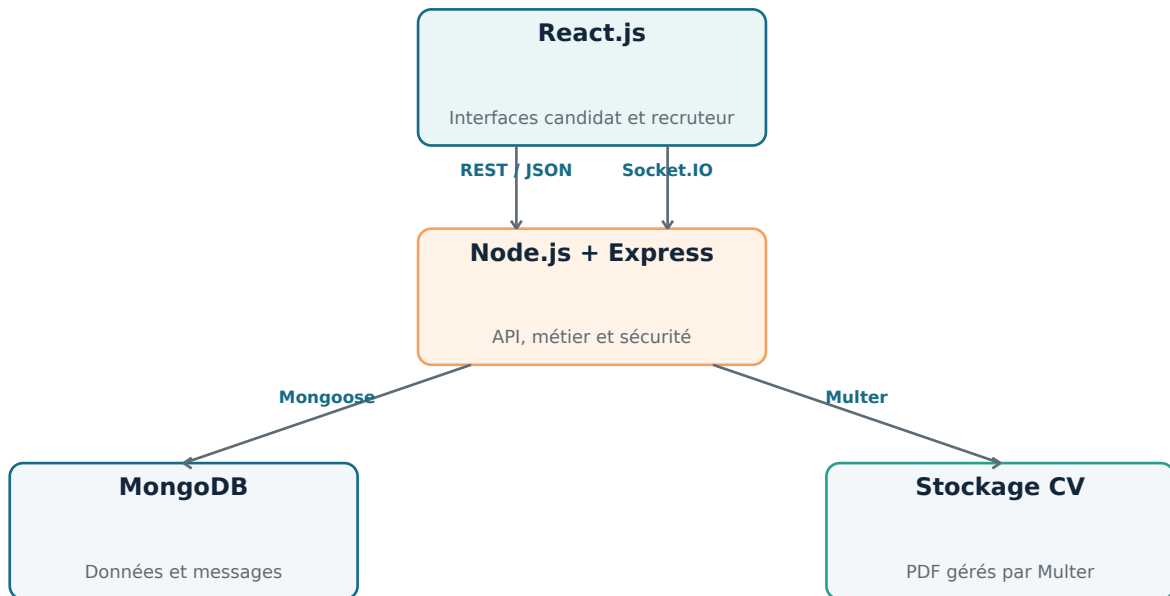
ID	Règle	Conséquence technique
RM-01	Chaque utilisateur possède un rôle unique : candidate ou recruter.	Middleware d'autorisation appliqué aux routes et événements.
RM-02	Les entreprises sont précréées dans le MVP.	Le recruteur sélectionne ou reçoit une entreprise existante ; pas de workflow admin.
RM-03	Seule une offre publiée est visible et peut recevoir un intérêt.	Contrôle du statut avant toute action candidat.
RM-04	L'intérêt candidat est unique pour le couple candidat-offre.	Index unique et traitement idempotent.
RM-05	Le recruteur ne décide que sur les intérêts de ses propres offres.	Vérification de propriété via l'offre et l'entreprise.
RM-06	Deux intérêts positifs créent un match unique.	Transaction logique et contrainte d'unicité.
RM-07	Le match crée ou rend disponible une conversation unique.	Aucune messagerie avant le match.
RM-08	Seuls les deux participants lisent et écrivent dans la conversation.	Contrôle d'appartenance avant join, lecture et envoi.
RM-09	Le CV est un PDF contrôlé et privé.	Accès via une route protégée, jamais par un répertoire public libre.

Scénario nominal du match



5. Architecture technologique retenue

Une stack JavaScript cohérente de bout en bout, limitée aux composants nécessaires.



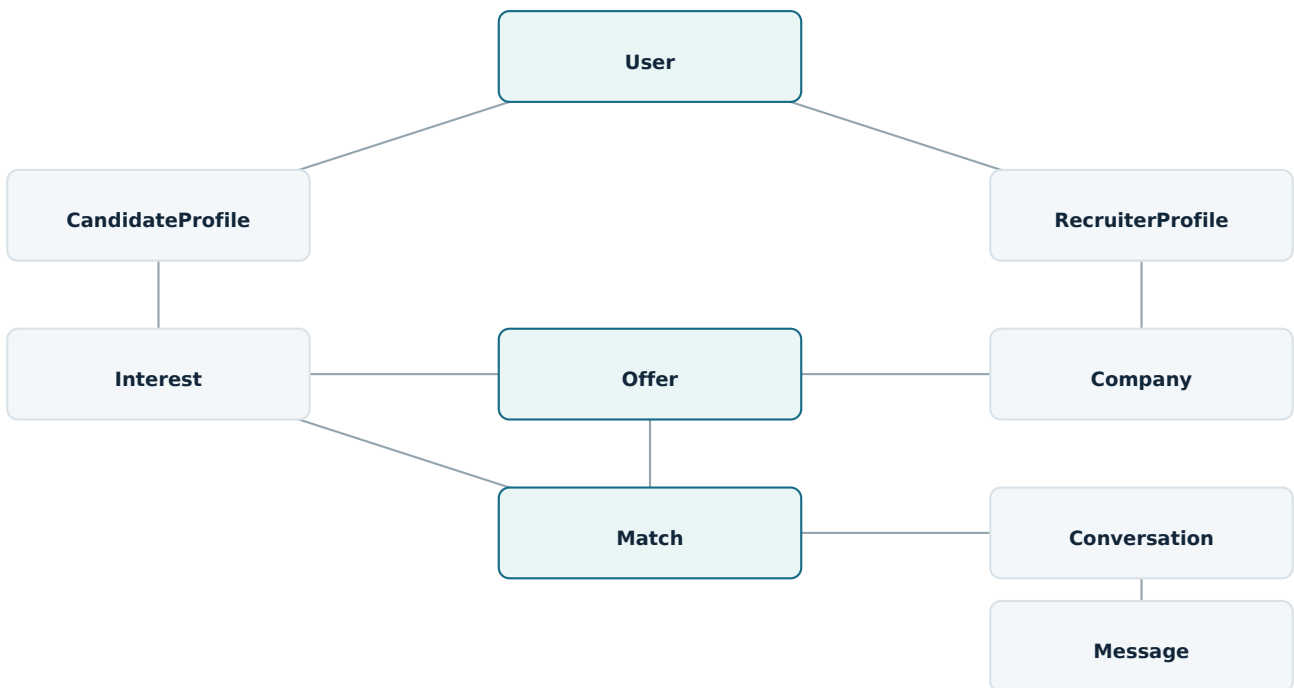
Couche	Technologie	Responsabilité
Interface	React.js	Écrans, navigation, formulaires, cartes de découverte et état local.
API	Node.js + Express.js	Routes REST, validation, logique métier, autorisation et orchestration.
Accès aux données	Mongoose	Schémas, validations, index, requêtes et relations par ObjectId.
Base	MongoDB	Utilisateurs, profils, entreprises, offres, intérêts, matchs, conversations et messages.
Temps réel	Socket.IO	Connexion bidirectionnelle, rooms par conversation et événements de messages.
Fichiers	Multer	Réception multipart, filtres, limite de taille et écriture des CV PDF.
Sécurité	JWT + bcrypt	Identité de session, contrôle des rôles et hachage des mots de passe.
Collaboration	Git + GitHub	Versionnement, branches, revues et historique des contributions.

Choix assumé

La première version reste en JavaScript afin de limiter la charge d'apprentissage. TypeScript, NestJS, microservices, files de messages ou services temps réel externes ne seront ajoutés que si un besoin prouvé apparaît.

6. Modèle de données conceptuel

Des collections séparées pour isoler l'authentification, les profils et les événements métier.



Collection	Champs structurants	Rôle
users	email, passwordHash, role	Identité et authentification.
candidateProfiles	userId, identité, titre, skills, experiences, educations, languages, cv	Dossier professionnel du candidat.
recruiterProfiles	userId, companyId, fonction	Profil métier du recruteur.
companies	nom, secteur, localisation, description	Entreprises de référence précréées.
offers	companyId, recruiterId, title, description, skills, status	Offres publiées ou fermées.
interests	candidateId, offerId, candidateLiked, recruiterLiked	État de l'intérêt réciproque.
matches	interestId, candidateId, offerId, recruiterId	Résultat métier de la réciprocité.
conversations	matchId, participantIds	Canal privé rattaché au match.
messages	conversationId, senderId, content, createdAt, readAt	Historique paginé des échanges.

Pourquoi séparer les messages ?

Les messages sont stockés dans une collection indépendante plutôt que dans un grand tableau embarqué dans la conversation. Cette structure facilite la pagination, la croissance de l'historique et les requêtes par date.

7. Messagerie temps réel

REST charge l'historique, Socket.IO transporte les événements, MongoDB garantit la persistance.

Architecture hybride retenue

Canal	Utilisation	Exemples
API REST	Données durables et chargement initial	Lister les conversations ; récupérer les messages par pages.
Socket.IO	Événements immédiats	Envoyer un message ; recevoir un message ; rejoindre une room.
MongoDB	Source de vérité	Conserver les messages avant leur diffusion.

Séquence d'envoi



- Chaque conversation correspond à une room nommée à partir de son identifiant.
- Le serveur vérifie l'appartenance à la conversation avant d'autoriser socket.join().
- Le message est enregistré avant d'être diffusé : ce qui est reçu en temps réel existe déjà en base.
- Le client ne fournit pas l'identité de l'expéditeur comme autorité ; celle-ci vient du JWT validé.
- Une reconnexion recharge l'historique par REST, évitant de dépendre uniquement des événements en direct.

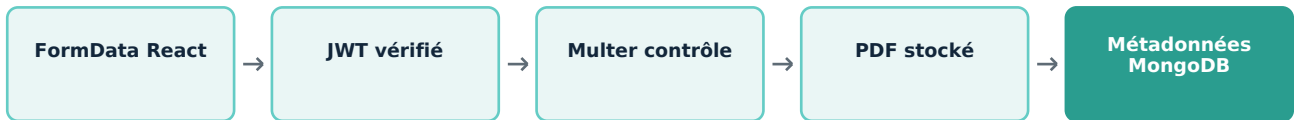
Événements minimaux

Événement	Direction	But
conversation:join	Client → serveur	Demander l'entrée dans une room après contrôle.
message:send	Client → serveur	Valider et enregistrer un nouveau message.
message:new	Serveur → room	Diffuser le message sauvegardé aux participants.
message:error	Serveur → client	Retourner un refus ou une erreur exploitable.

8. Gestion des CV avec Multer

Un fichier privé sur disque pour le MVP ; des métadonnées légères en base.

Flux d'upload



Contrôle	Décision MVP	Risque couvert
Authentification	Appliquée avant Multer	Dépôt anonyme de fichiers.
Champ	upload.single('cv')	Nombre ou nom de fichiers inattendu.
Format	PDF uniquement ; MIME + extension	Fichiers manifestement non conformes.
Taille	5 Mo maximum	Surcharge disque/mémoire et abus simples.
Nom	Identifiant utilisateur + horodatage	Collision et noms dangereux fournis par le client.
Stockage	uploads/cvs sur le serveur	Implémentation simple et démontrable du MVP.
Base	filename, path, mimetype, size	Évite de stocker le binaire directement dans MongoDB.
Consultation	Route API protégée	Exposition publique des CV.
Remplacement	Nouveau fichier validé puis ancien supprimé	Accumulation de fichiers orphelins.

Limites connues et évolution

Le type MIME annoncé par le navigateur n'est pas une preuve absolue. Une version de production devrait vérifier la signature réelle du fichier, analyser les contenus malveillants, isoler le stockage et migrer vers un stockage objet privé. L'interface de données reste compatible : le champ path devient une URL ou une clé de stockage.

9. Sécurité et contrôle d'accès

Authentifier ne suffit pas : chaque ressource fait aussi l'objet d'une autorisation métier.

Mécanisme	Fonction	Exemple dans le projet
bcrypt	Hacher le mot de passe	Aucun mot de passe en clair en base.
JWT	Identifier l'utilisateur	API REST et handshake Socket.IO.
RBAC	Limiter selon le rôle	Seul un recruteur publie une offre.
Contrôle de propriété	Vérifier l'accès à la ressource	Le recruteur agit uniquement sur ses offres.
Contrôle de participation	Protéger une conversation	Seuls le candidat et le recruteur du match entrent dans la room.
Validation serveur	Ne pas faire confiance au client	Statut de l'offre, identifiants et contenu du message vérifiés.
Limites	Réduire les abus	Taille du CV, longueur du message, pagination et débit.

Matrice d'autorisation simplifiée

Action	Candidat	Recruteur	Condition
Consulter les offres publiées	Oui	Oui	Utilisateur authentifié.
Publier / fermer une offre	Non	Oui	Offre appartenant au recruteur.
Exprimer un intérêt	Oui	Non	Offre publiée et intérêt non existant.
Évaluer un candidat	Non	Oui	Candidat intéressé par une offre détenue.
Lire un CV	Propriétaire	Oui	Accès métier au profil concerné.
Rejoindre une conversation	Oui	Oui	Participant du match correspondant.
Envoyer un message	Oui	Oui	Conversation autorisée et contenu valide.

Question de soutenance à maîtriser : authentification = “qui êtes-vous ?” ; autorisation = “avez-vous le droit d'effectuer cette action sur cette ressource ?”.

10. Contrats d'API et écrans attendus

Une première carte technique reliant directement fonctionnalités, routes et interfaces.

Routes REST indicatives

Méthode et route	Rôle	Résultat
POST /api/auth/register	Public	Créer un compte et son rôle.
POST /api/auth/login	Public	Recevoir un JWT.
GET /api/offers	Candidat	Lister les offres publiées.
POST /api/offers	Recruteur	Publier une offre.
PATCH /api/offers/:id/close	Recruteur	Fermer sa propre offre.
POST /api/offers/:id/interests	Candidat	Enregistrer son intérêt.
GET /api/offers/:id/interests	Recruteur	Lister les candidats intéressés.
PATCH /api/interests/:id/decision	Recruteur	Accepter ou refuser ; créer le match si nécessaire.
GET /api/conversations	Les deux	Lister les conversations autorisées.
GET /api/conversations/:id/messages	Les deux	Charger l'historique paginé.
POST /api/candidate/cv	Candidat	Téléverser ou remplacer le CV.
GET /api/candidates/:id/cv	Autorisé	Servir le PDF après contrôle.

Écrans du candidat

Connexion / inscription ; tableau de bord ; profil ; téléversement du CV ; découverte des offres ; détail d'une offre ; matchs ; conversations ; messagerie.

Écrans du recruteur

Connexion / inscription ; tableau de bord ; profil et entreprise ; liste des offres ; création d'une offre ; détail et fermeture ; candidats intéressés ; évaluation d'un profil ; matchs ; conversations ; messagerie.

11. Plan de réalisation recommandé

Construire d'abord une tranche verticale fonctionnelle, puis enrichir et sécuriser.

Lot	Objectif	Résultat démontrable
0 - Socle	Dépôt Git, conventions, variables d'environnement, connexion MongoDB	L'application démarre de manière reproductible.
1 - Identité	Inscription, connexion, JWT, rôles	Deux utilisateurs peuvent se connecter dans leurs espaces.
2 - Profils	Profils candidat/recruteur et entreprises précréées	Les données métier principales sont visibles et modifiables.
3 - Offres	Créer, lister, consulter et fermer	Un recruteur publie ; un candidat découvre.
4 - Intérêts	Intérêt candidat et file de profils par offre	Le recruteur voit les candidats réellement intéressés.
5 - Match	Décision recruteur, unicité et conversation	La boucle métier centrale est complète.
6 - Messagerie	Historique REST puis temps réel Socket.IO	Deux participants échangent sans actualiser.
7 - CV	Upload Multer et consultation protégée	Un candidat dépose son CV ; un recruteur autorisé le lit.
8 - Qualité	Validation, erreurs, tests, responsive, démonstration	Version finale stable et explicable.

Organisation d'équipe conseillée

Responsabilité	Mission	Preuve attendue
Coordination	Backlog, priorités, points d'avancement et arbitrages	Tableau de suivi à jour.
Référent technique	Architecture, conventions et intégration	Décisions documentées et code relu.
Frontend	Parcours, composants et responsive	Écrans reliés aux cas d'utilisation.
Backend / données	API, schémas, sécurité et temps réel	Routes testées et règles métier centralisées.
Documentation / démo	Cahier des charges, rapports et scénario	Chaque membre sait expliquer sa contribution.

12. Risques, arbitrages et critères de réussite

Les principaux risques sont identifiés avant le développement afin de protéger la démonstration finale.

Risque	Impact	Réponse prévue
Ajout continu de fonctionnalités	MVP inachevé	Backlog V2 séparé et règle : aucun ajout avant la boucle complète.
Messagerie commencée trop tôt	Blocage technique	Historique REST d'abord, puis Socket.IO.
Autorisation gérée seulement dans React	Fuite de données	Contrôles systématiques dans Express et Socket.IO.
Doublons d'intérêts ou de matchs	Incohérence métier	Index uniques et opérations idempotentes.
CV exposés publiquement	Atteinte à la confidentialité	Route de téléchargement protégée.
Code généré non compris	Échec aux questions techniques	Revue en équipe, documentation et démonstration par chaque membre.
Démonstration dépendante d'Internet	Démo instable	Jeu de données local, comptes de test et scénario répété.

Définition de terminé pour le MVP

- La boucle candidat → intérêt → décision recruteur → match → message fonctionne de bout en bout.
- Les deux rôles voient des écrans et permissions cohérents.
- Les données persistent après redémarrage et l'historique des messages est récupérable.
- Les CV PDF sont limités, enregistrés et servis sous autorisation.
- Les erreurs critiques sont gérées et les cas interdits sont testés.
- Le dépôt, le modèle de données, les routes et les choix techniques sont documentés.
- Chaque membre peut expliquer l'architecture et sa propre contribution.

13. Feuille de route et préparation de la soutenance

Le dossier répond aux attendus du module : comprendre, concevoir, développer, expliquer et présenter.

Jalons

Jalon	Livrable	Validation
Initialisation	Project Charter : nom, équipe, description, stack, fonctions	Périmètre et responsabilités acceptés.
Conception	Cahier des charges, parcours, règles, modèle et API	Cas d'utilisation sans ambiguïté.
Prototype	Authentification + première tranche offre/intérêt	Une fonction réelle et une architecture visible.
MVP	Boucle complète + messagerie + CV	Scénario nominal démontrable.
Finalisation	Tests, données de démo, documentation et répétition	Version stable et équipe prête aux questions.

Répartition des 20 minutes

Temps	Partie	Contenu recommandé
5 min	Présentation	Besoin, valeur, utilisateurs, MVP et justification de la stack.
10 min	Démonstration	Scénario complet : candidat intéressé, recruteur accepte, match, message, CV.
5 min	Questions	Architecture, MongoDB, sécurité, Socket.IO, Multer, difficultés et usage de l'IA.

Questions techniques à préparer

- Pourquoi MongoDB malgré les nombreuses relations entre les objets ?
- Pourquoi REST et Socket.IO sont-ils complémentaires ?
- Comment empêchez-vous un utilisateur d'entrer dans une conversation tierce ?
- Pourquoi stocker les messages séparément des conversations ?
- Comment sécurisez-vous et remplacez-vous un CV ?
- Qu'avez-vous volontairement exclu du MVP, et pourquoi ?
- Comment l'IA vous a-t-elle aidés sans remplacer votre compréhension ?

14. Décision finale et prochaines actions

Le concept et la stack sont arrêtés ; l'équipe peut passer à la conception détaillée puis au développement.

Architecture cible : React.js + Node.js/Express + MongoDB/Mongoose + JWT/bcrypt + Socket.IO + Multer + Git/GitHub.

Actions immédiates

	Valider le nom du projet et la composition de l'équipe.
	Transformer ce dossier en Project Charter d'une page et en backlog priorisé.
	Figurer les maquettes des écrans MVP et les critères d'acceptation.
	Détailler les schémas Mongoose, index uniques et contrats d'API.
	Initialiser le dépôt GitHub et réaliser le lot 0 puis le lot 1.
	Préparer dès le début un jeu de données et un scénario de démonstration.

Glossaire

Terme	Définition dans ce projet
Intérêt	Action positive exprimée par le candidat pour une offre, puis par le recruteur pour ce candidat.
Match	Résultat unique d'un intérêt positif réciproque sur une offre.
Conversation	Canal privé créé à partir d'un match et réservé à ses deux participants.
MVP / PMV	Produit minimum viable : plus petite version réalisant la boucle métier complète.
Room	Groupe Socket.IO représentant une conversation autorisée.
ODM	Couche de modélisation objet-document ; ici Mongoose entre Node.js et MongoDB.

Référence de cadrage : support du module « Projet Web Collaboratif et Remise à Niveau Technique », modifié le 9 septembre 2026, ainsi que les décisions consolidées lors des échanges de l'équipe.